

Pana ERP v3.0 - Architecture & Implementation Guide

Document Version: 3.0.3
Last Updated: 2025-12-30
Status: PRODUCTION READY
Golden Template: Items Module (Stock)

Table of Contents

- 1. [Executive Summary](#)
- 2. [Core Philosophy](#)
- 3. [Technology Stack](#)
- 4. [Schema-First Type System](#)
- 5. [Centralized Configuration](#)
- 6. [Factory Pattern Architecture](#)
- 7. [Directory Structure](#)
- 8. [Component Architecture](#)
- 9. [API Layer](#)
- 10. [State Management](#)
- 11. [Styling System](#)
- 12. [Theme System](#)
- 13. [Error Handling](#)
- 14. [Module Creation Workflow](#)
- 15. [Known Issues & Workarounds](#)
- 16. [Changelog](#)

1. Executive Summary

1.1 Strategic Decision

Pana ERP v3.0 represents a fundamental architectural shift from "Page-First" development to **"Schema-First"** development. This decision was made to:

- **Eliminate type drift** between Frappe backend and Next.js frontend
- **Reduce boilerplate** by 70%+ through generic factories
- **Enable rapid module development** (target: new module in <2 hours)
- **Ensure runtime type safety** via auto-generated Zod schemas
- **Maintain premium UI** with dual-theme (light/dark) support

1.2 Key Mandates

Mandate	Description
Zero Manual Types	All DocType interfaces MUST be generated from Frappe metadata
Factory Pattern	All CRUD operations MUST use generic hooks and route handlers

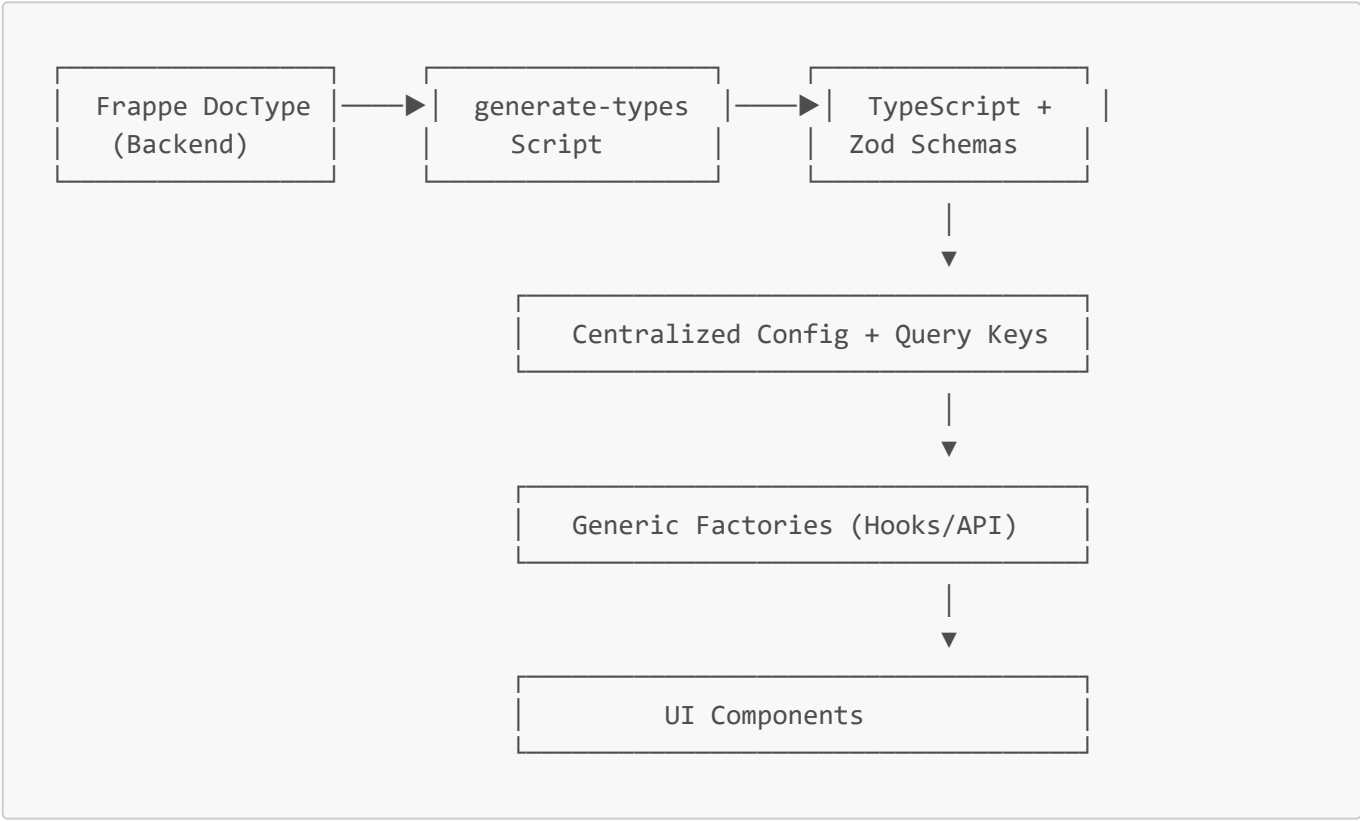
Mandate	Description
Single Source of Truth	<code>types/doctype-types.ts</code> for types, <code>lib/doctype-config.ts</code> for config
Theme Awareness	All components MUST use theme-aware CSS variables
Premium UI	Maintain "Big Tech" aesthetic with animations and responsive design

1.3 Golden Template

The **Items Module** (`app/stock/item/`) is the canonical reference implementation. All new modules MUST follow its patterns.

2. Core Philosophy

2.1 Schema-Driven Development Flow



2.2 Anti-Patterns (DO NOT DO)

```
// ✗ NEVER define manual interfaces for DocTypes
interface Item {
  name: string;
  item_name: string;
}

// ✗ NEVER write custom hooks for standard CRUD
export function useItemsQuery() { ... }

// ✗ NEVER hardcode colors
```

```
className="bg-white" // Use bg-card instead

// ✗ NEVER duplicate doctypeToApiPath mappings
function doctypeToApiPath() { ... } // Use getApiPath() from doctype-config
```

2.3 Correct Patterns (ALWAYS DO)

```
// ☒ Import from generated types
import { Item } from "@types/doctype-types";

// ☒ Use generic hooks
const { data } = useFrappeList<Item>("Item", { filters: [...] });

// ☒ Use factory handlers for API routes
export const GET = createListHandler("Item");

// ☒ Use centralized config
import { getApiPath } from "@lib/doctype-config";

// ☒ Use theme-aware colors
className="bg-card"
```

3. Technology Stack

3.1 Core Dependencies

Layer	Technology	Version	Purpose
Framework	Next.js	15.x (App Router)	Full-stack React framework
Language	TypeScript	5.x (Strict Mode)	Type safety
Styling	Tailwind CSS	v4.x	Utility-first CSS
State	TanStack Query	v5.x	Server state management
Forms	React Hook Form	v7.x	Form handling
Validation	Zod	v3.x	Runtime type validation
Backend	Frappe Framework	v15	REST API provider
Icons	Lucide React	Latest	Consistent iconography
Animations	Framer Motion	v12.x	Micro-interactions

3.2 Frappe JS SDK

```
import { FrappeApp } from "frappe-js-sdk";
```

```
const frappe = new FrappeApp(apiUrl, {
  useToken: true,
  token: () => `${apiKey}:${apiSecret}`,
  type: "token",
});
```

4. Schema-First Type System

4.1 Type Generation

Location: `scripts/generate-types.js`

Usage:

```
# Generate types for specific DocTypes
pnpm generate-types Item Customer SalesOrder

# Regenerate all known DocTypes
pnpm generate-types --all
```

4.2 Generated Output

`types/doctype-types.ts`:

```
// AUTO-GENERATED FILE - DO NOT EDIT MANUALLY

export interface Item {
  name: string;
  item_code: string;
  item_name?: string;
  item_group: string;
  stock_uom: string;
  is_stock_item?: 0 | 1;
  description?: string;
  disabled?: 0 | 1;
  // ... more fields
  creation?: string;
  modified?: string;
}

export type ItemCreateRequest = Pick<Item, "item_code" | "item_group" |
"stock_uom"> &
  Partial<Pick<Item, "item_name" | "description" | "disabled">>;

export type ItemUpdateRequest = Partial<Omit<Item, "name" | "creation">>;
```

4.3 Frappe Field Type Mapping

Frappe Type	TypeScript Type	Zod Schema
Data	string	z.string()
Link	string	z.string()
Select	string	z.enum([...])
Int	number	z.number().int()
Float/Currency	number	z.number()
Check	0 \ 1	z.union([z.literal(0), z.literal(1)])
Text/Long Text	string	z.string()
Date/Datetime	string	z.string() (ISO format)
Table	ChildDocType[]	z.array(ChildSchema)

5. Centralized Configuration

5.1 DocType Configuration Registry

Location: lib/doctype-config.ts

This is the **SINGLE SOURCE OF TRUTH** for all DocType metadata:

```
export const DOCTYPE_CONFIG: Record<string, DocTypeConfig> = {
  Item: {
    apiPath: "stock/item",
    module: "Stock",
    labelField: "item_name",
    searchFields: ["item_code", "item_name", "description"],
    defaultSortField: "creation",
    defaultSortOrder: "desc",
  },
  "Item Group": {
    apiPath: "stock/settings/item-group",
    module: "Stock",
    labelField: "item_group_name",
    isSettings: true,
  },
  Customer: {
    apiPath: "crm/customer",
    module: "CRM",
    labelField: "customer_name",
    searchFields: ["customer_name", "email_id"],
  },
  // ... all doctypes
};

// Utility functions
export function getApiPath(doctype: string): string;
```

```
export function getLabelField(doctype: string): string;
export function getSearchFields(doctype: string): string[];
export function getDocTypesByModule(module: ModuleCategory): string[];
```

When adding a new module, ALWAYS add the DocType here first.

5.2 Query Key Factory

Location: lib/query-keys.ts

Ensures consistent cache key generation:

```
export const queryKeys = {
  item: {
    all: () => ["Item"] as const,
    list: (options?: FrappeListOptions) => ["Item", "list", options] as const,
    doc: (name: string) => ["Item", "doc", name] as const,
  },
  customer: {
    all: () => ["Customer"] as const,
    list: (options?: FrappeListOptions) => ["Customer", "list", options] as const,
    doc: (name: string) => ["Customer", "doc", name] as const,
  },
  // ... all doctypes
};

// For dynamic doctypes
export function getQueryKeys(doctype: string) {
  return {
    all: () => [doctype] as const,
    list: (options?: FrappeListOptions) => [doctype, "list", options] as const,
    doc: (name: string) => [doctype, "doc", name] as const,
  };
}
```

6. Factory Pattern Architecture

6.1 Generic Hooks

Location: hooks/generic/

Hook	Purpose	Example
useFrappeList<T>	Fetch list with filters/pagination	useFrappeList<Item>("Item", { limit: 50 })
useFrappeDoc<T>	Fetch single document	useFrappeDoc<Item>("Item", "ITEM-001")
useFrappeCreate<T>	Create new document	useFrappeCreate<Item>("Item")

Hook	Purpose	Example
<code>useFrappeUpdate<T></code>	Update existing document	<code>useFrappeUpdate<Item>("Item")</code>
<code>useFrappeDelete</code>	Delete document	<code>useFrappeDelete("Item")</code>
<code>useFrappeOptions</code>	Fetch dropdown options	<code>useFrappeOptions("Item Group", "name")</code>

Usage:

```
import { useFrappeList, useFrappeCreate, useFrappeDelete } from "@/hooks/generic";
import type { Item, ItemCreateRequest } from "@/types/doctype-types";

// List
const { data: items, isLoading } = useFrappeList<Item>("Item", {
  filters: [{"item_group", "=", "Raw Materials"}],
  orderBy: { field: "creation", order: "desc" },
  limit: 50,
});

// Create
const createMutation = useFrappeCreate<{ data: Item }, ItemCreateRequest>("Item", {
  onSuccess: () => router.push("/stock/item"),
});

// Delete
const deleteMutation = useFrappeDelete("Item", {
  onSuccess: () => setDeleteTarget(null),
});
```

6.2 API Factory

Location: `lib/api-factory.ts`

Factory	HTTP Method	Usage
<code>createListHandler</code>	GET	List with filters/pagination
<code>createGetHandler</code>	GET	Fetch single document
<code>createCreateHandler</code>	POST	Create new document
<code>createUpdateHandler</code>	PUT	Update document
<code>createDeleteHandler</code>	DELETE	Delete document

Usage:

```
// app/api/stock/item/route.ts
import { createListHandler, createCreateHandler } from "@/lib/api-factory";
```

```
export const GET = createListHandler("Item", {
  allowedFields: ["name", "item_code", "item_name", "item_group"],
  defaultSort: { field: "creation", order: "desc" },
});

export const POST = createCreateHandler("Item");

// app/api/stock/item/[name]/route.ts
import { createGetHandler, createUpdateHandler, createDeleteHandler } from
"@/lib/api-factory";

export const GET = createGetHandler("Item");
export const PUT = createUpdateHandler("Item");
export const DELETE = createDeleteHandler("Item");
```

7. Directory Structure

```

pana-erp/
├── app/
│   ├── api/
│   │   ├── {module}/
│   │   │   ├── {doctype}/
│   │   │   │   ├── route.ts
│   │   │   │   └── [name]/
│   │   │   │       └── route.ts
│   │   └── {module}/
│   │       ├── {doctype}/
│   │       │   ├── page.tsx
│   │       │   ├── new/
│   │       │   │   └── page.tsx
│   │       │   ├── [name]/
│   │       │   │   ├── page.tsx
│   │       │   │   └── edit/
│   │       │   │       └── page.tsx
│   └── {module}/
│       ├── {doctype}/
│       │   ├── page.tsx
│       │   ├── new/
│       │   │   └── page.tsx
│       │   ├── [name]/
│       │   │   ├── page.tsx
│       │   │   └── edit/
│       │   │       └── page.tsx
│       └── {doctype}/
│           ├── page.tsx
│           ├── new/
│           │   └── page.tsx
│           ├── [name]/
│           │   ├── page.tsx
│           │   └── edit/
│           │       └── page.tsx
├── components/
│   ├── ui/
│   │   ├── button.tsx
│   │   ├── input.tsx
│   │   ├── select.tsx
│   │   └── ...
│   └── smart/
│       ├── page-header.tsx
│       ├── frappe-select.tsx
│       ├── searchable-select.tsx
│       ├── data-field.tsx
│       ├── confirm-dialog.tsx
│       └── print-label-dialog.tsx

```

```

├── empty-state.tsx
├── loading-state.tsx
├── status-badge.tsx
├── theme-toggle.tsx
├── form/                                # Form Field Wrappers
│   ├── form-input.tsx
│   ├── form-textarea.tsx
│   ├── form-select.tsx
│   ├── form-switch.tsx
│   └── form-frappe-select.tsx
├── Layout/                             # Layout Components
│   ├── Layout.tsx
│   └── QueryProvider.tsx
├── hooks/
│   ├── generic/                        # Generic Frappe Hooks
│   │   ├── useFrappeList.ts
│   │   ├── useFrappeDoc.ts
│   │   ├── useFrappeMutation.ts
│   │   ├── useFrappeOptions.ts
│   │   └── index.ts
│   └── useDeleteWithConfirmation.ts    # Reusable Delete Hook
├── lib/
│   ├── doctype-config.ts              # DocType Registry (CRITICAL)
│   ├── query-keys.ts                  # Query Key Factory
│   ├── api-factory.ts                 # API Route Factories
│   ├── frappe-client.ts               # Frappe SDK Wrapper
│   ├── theme-context.tsx              # Theme Provider
│   ├── utils.ts                       # Utilities (cn, etc.)
│   └── schemas/
│       ├── doctype-schemas.ts        # Generated Zod Schemas
│       └── form-helpers.ts            # Form Utilities
├── types/
│   └── doctype-types.ts               # Generated TypeScript Interfaces
├── docs/
│   ├── v3/
│   │   └── ARCHITECTURE_V3.md         # This Document
│   ├── MODULE_CREATION_WORKFLOW.md    # Step-by-Step Guide
│   ├── THEME_SYSTEM.md                # Theme Documentation
│   └── TEMPLATE_REVIEW_V3.md          # Template Analysis
├── scripts/
│   └── generate-types.js              # Type Generation Script

```

8. Component Architecture

8.1 Smart Components

Location: `components/smart/`

Component	Purpose
PageHeader	Floating header with title, search, actions
FrappeSelect	Async dropdown fetching from Frappe DocType
SearchableSelect	Searchable dropdown with scroll
DataField	Form field wrapper with label/error
ConfirmDialog	Premium delete confirmation
PrintLabelDialog	Label printing with preview
EmptyState	No data display
LoadingState	Loading skeletons
StatusBadge	Colored status indicator
ThemeToggle	Light/Dark/System toggle

Usage:

```
import {
  PageHeader,
  FrappeSelect,
  ConfirmDialog,
  EmptyState,
  LoadingState,
} from "@components/smart";
```

8.2 Form Components

Location: `components/form/`

Component	Purpose
FormInput	Text input with DataField wrapper
FormTextarea	Textarea with consistent styling
FormSelect	Static select dropdown
FormSwitch	Toggle switch with label
FormFrappeSelect	Async select from Frappe

Usage:

```
import { FormInput, FormFrappeSelect, FormSwitch } from "@components/form";

<FormInput
  control={form.control}
  name="item_name"
  label="Item Name"
  required
  placeholder="Enter item name..."
/>

<FormFrappeSelect
  control={form.control}
  name="item_group"
  label="Item Group"
  required
  doctype="Item Group"
  labelField="item_group_name"
/>

<FormSwitch
  control={form.control}
  name="is_stock_item"
  label="Maintain Stock"
  description="Track stock levels for this item"
/>
```

9. API Layer

9.1 Route Structure

```
/api/{module}/{doctype}/          → GET (list), POST (create)
/api/{module}/{doctype}/{name}/    → GET (single), PUT (update), DELETE
```

9.2 Response Format

Success:

```
{
  "success": true,
  "data": { /* document or array */ },
  "message": "Optional success message"
}
```

Error:

```
{
  "success": false,
  "error": "Error Type",
  "details": "Detailed error message or validation errors"
}
```

9.3 Query Parameters

Parameter	Description	Example
fields	JSON array of fields to fetch	["name", "item_code"]
filters	JSON array of filter tuples	[["item_group", "=", "Products"]]
order_by	Sort field and direction	creation desc
limit	Number of records	50
offset	Pagination offset	0
search	Search term	widget

10. State Management

10.1 TanStack Query Configuration

```
// lib/query-client.ts
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000,      // 1 minute
      gcTime: 5 * 60 * 1000,    // 5 minutes
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
});
```

10.2 Cache Invalidation

Mutations automatically invalidate related queries:

```
// After create/update/delete
queryClient.invalidateQueries({ queryKey: [doctype] });
```

11. Styling System

11.1 CSS Variables

All colors use CSS variables for theme support:

```
/* Light Mode (default) */
:root {
  --color-background: oklch(0.98 0.01 240);
  --color-foreground: oklch(0.15 0.02 240);
  --color-card: oklch(1 0 0);
  --color-primary: oklch(0.2 0.02 240);
  /* ... */
}

/* Dark Mode */
.dark {
  --color-background: oklch(0.15 0.015 240);
  --color-foreground: oklch(0.95 0.005 240);
  --color-card: oklch(0.19 0.02 240);
  --color-primary: oklch(0.55 0.18 265);
  /* ... */
}
```

11.2 Color Usage Rules

Use Case	Light Mode	Dark Mode	Tailwind Class
Page background	Off-white	Dark gray	bg-background
Cards/panels	White	Elevated gray	bg-card
Popovers/dropdowns	White/opacity	Elevated gray	bg-popover
Form inputs (focus)	White	Card color	focus:bg-card
Borders	Light gray	Visible gray	border-border
Text	Near black	Near white	text-foreground

NEVER use **bg-white** - use **bg-card** instead!

12. Theme System

12.1 ThemeProvider

Location: lib/theme-context.tsx

```
// Wrap your app with ThemeProvider
<ThemeProvider defaultTheme="system">
  <App />
</ThemeProvider>
```

```
// Use the theme in components
const { theme, setTheme, resolvedTheme } = useTheme();
```

12.2 Theme Options

Theme	Behavior
light	Always light mode
dark	Always dark mode
system	Follow OS preference

12.3 ThemeToggle Component

```
import { ThemeToggle, ThemeToggleButton } from "@components/smart";

// Dropdown with all options
<ThemeToggle />

// Simple toggle button
<ThemeToggleButton />
```

13. Error Handling

13.1 API Errors

All API factories return consistent error responses:

```
return NextResponse.json({
  success: false,
  error: "Operation Failed",
  details: error.message,
}, { status: 500 });
```

13.2 Form Validation Errors

Zod schemas provide field-level errors:

```
if (!result.success) {
  return NextResponse.json({
    success: false,
    error: "Validation Error",
    details: result.error.flatten().fieldErrors,
  });
}
```

```
    }, { status: 400 });  
  }
```

13.3 Toast Notifications

Mutations automatically show toasts via Sonner:

```
// Success  
toast.success("Item created successfully");  
  
// Error  
toast.error("Failed to create Item", {  
  description: error.message,  
});
```

14. Module Creation Workflow

See [docs/MODULE_CREATION_WORKFLOW.md](#) for complete step-by-step guide.

Quick Checklist

```
[ ] 1. Generate types: pnpm generate-types "DocType Name"  
[ ] 2. Add to lib/doctype-config.ts  
[ ] 3. Add to lib/query-keys.ts  
[ ] 4. Create API routes (use factories)  
[ ] 5. Create List page  
[ ] 6. Create New page  
[ ] 7. Create Detail page  
[ ] 8. Create Edit page  
[ ] 9. Test CRUD operations  
[ ] 10. Test dark mode  
[ ] 11. Test responsive design
```

15. Known Issues & Workarounds

15.1 React Hook Form + Zod Type Inference

Issue: Complex type inference causes false positive errors.

Solution: Add `@ts-nocheck` at top of form pages:

```
// @ts-nocheck - React Hook Form + Zod type inference limitations
```

15.2 Next.js 15+ Async Params

Issue: Route params are now `Promise<{ name: string }>`.

Solution:

```
export async function GET(
  request: NextRequest,
  { params }: { params: Promise<{ name: string }> }
) {
  const { name } = await params;
  // ...
}
```

15.3 Boolean Field Handling

Issue: Frappe uses `0/1`, React uses `true/false`.

Solution: Convert in form transformers:

```
// Form to Frappe
const frappeData = { ...formData, disabled: formData.disabled ? 1 : 0 };

// Frappe to Form
const formData = { ...frappeData, disabled: frappeData.disabled === 1 };
```

16. Changelog

v3.0.3 (2025-12-30) 🏗️ ARCHITECTURE FINALIZATION

Major Architectural Improvements:

- **Centralized DocType Configuration** (`lib/doctype-config.ts`)
 - Single source of truth for all DocType metadata
 - Eliminated duplicate `doctypeToApiPath` functions
 - Utility functions: `getApiPath()`, `getLabelField()`, `getSearchFields()`
- **Query Key Factory** (`lib/query-keys.ts`)
 - Consistent cache key generation
 - Prevents cache key mismatches
- **Form Schema Helpers** (`lib/schemas/form-helpers.ts`)
 - Common field schemas
 - Form-to-Frappe data converters

- **Delete Confirmation Hook** (`hooks/useDeleteWithConfirmation.ts`)
 - Reusable delete logic with `ConfirmDialog` integration
- **Form Components** (`components/form/`)
 - `FormInput`, `FormTextarea`, `FormSelect`, `FormSwitch`, `FormFrappeSelect`
 - Reduces form boilerplate by ~50%

Documentation:

- Created `docs/MODULE_CREATION_WORKFLOW.md`
 - Created `docs/TEMPLATE_REVIEW_V3.md`
-

v3.0.2 (2025-12-30) 🐛 THEME SYSTEM

Dual Theme System:

- `ThemeProvider` context with light/dark/system modes
- OKLCH color system for perceptual uniformity
- `localStorage` persistence
- Smooth 200ms transitions
- `.transitioning` class for animation control

Color System Improvements:

- Fixed 50+ instances of `bg-white` → `bg-card`
- Fixed dropdowns, dialogs, form inputs
- Fixed sidebar icons visibility in dark mode

SearchableSelect Component:

- Custom implementation (removed `cmdk` dependency)
 - Fixed mouse click selection
 - Added search and scroll support
-

v3.0.1 (2025-12-29) 🚀 INITIAL RELEASE

New Features:

- Schema-driven type generation
- Factory patterns for API routes
- Generic CRUD hooks
- Smart components (`PageHeader`, `FrappeSelect`, etc.)
- `ConfirmDialog` and `PrintLabelDialog`

Migrations:

- Migrated Items Module to v3.0 patterns
- Migrated Item Group, Item Price, UOM APIs

Code Reduction:

Area	Reduction
API Routes	~75%
Form Boilerplate	~50%
Total per Module	~60%

Summary

Pana ERP v3.0 is a **production-ready boilerplate** with:

- 1. ☒ **Schema-driven types** from Frappe metadata
- 2. ☒ **Centralized configuration** for all DocTypes
- 3. ☒ **Factory patterns** for API routes and hooks
- 4. ☒ **Reusable form components** with consistent styling
- 5. ☒ **Premium UI** with dark mode support
- 6. ☒ **Comprehensive documentation** for team development

Golden Template: Follow the Items module ([app/stock/item/](#)) for all new modules.

Reference Documents:

- [docs/MODULE_CREATION_WORKFLOW.md](#) - Step-by-step guide
- [docs/THEME_SYSTEM.md](#) - Theme implementation details
- [docs/TEMPLATE_REVIEW_V3.md](#) - Design decisions

This architecture document is the single source of truth for Pana ERP v3.0 development.

Last reviewed and finalized: 2025-12-30